

УНИВЕРСИТЕТ ИТМО

Факультет прикладной информатики  
Направление подготовки 09.03.03 Прикладная информатика  
Дисциплина «Мобильная разработка (Android и iOS)»



Курсовая работа на тему:  
**«Разработка Android-приложения с модульной архитектурой на  
основе Clean Architecture и MVVM с поддержкой локального хранения и  
фоновой синхронизации»**

**Выполнили:**

Кузнецов А. Г. К3440

Скирляк Я. Ю. К3439

**Преподаватель:**

Шатинский Г. С.

Санкт-Петербург, 2025 г.

## СОДЕРЖАНИЕ

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>ВВЕДЕНИЕ .....</b>                                  | <b>3</b>  |
| <b>ОСНОВНАЯ ЧАСТЬ .....</b>                            | <b>4</b>  |
| 1      Проектирование приложения .....                 | 4         |
| 2      Разработка навигации .....                      | 6         |
| 3      Сетевое взаимодействие и работа с данными ..... | 8         |
| 4      Организация локального хранения данных .....    | 9         |
| 5      Разработка функциональных модулей .....         | 10        |
| 5.1      Авторизация пользователя .....                | 10        |
| 5.2      Профиль пользователя .....                    | 12        |
| 5.3      Настройки приложения .....                    | 15        |
| 5.4      Лента цитат .....                             | 16        |
| 6      Реализация фоновых задач и уведомлений .....    | 19        |
| <b>ЗАКЛЮЧЕНИЕ .....</b>                                | <b>22</b> |

## **ВВЕДЕНИЕ**

### **Цель работы:**

Создание Android-приложения, позволяющего управлять пользовательскими данными и обновлять список цитат из сети с возможностью работы в офлайн-режиме, соблюдая принципы модульности и разделения ответственности между слоями приложения.

### **Задачи работы:**

- 1) Спроектировать многомодульное приложение с разделением на Presentation, Domain и Data слои.
- 2) Реализовать навигацию между экранами с помощью Jetpack Compose Navigation.
- 3) Настроить обмен данными с внешним API с использованием корутин.
- 4) Организовать локальное хранение данных в Room и Proto DataStore.
- 5) Разработать функционал экранов «Лента», «Профиль» и «Настройки».
- 6) Настроить фоновую синхронизацию данных с уведомлениями через WorkManager.

## ОСНОВНАЯ ЧАСТЬ

### 1 Проектирование приложения

Для обеспечения высокой масштабируемости и упрощения сопровождения приложение было спроектировано с использованием многомодульного подхода. Архитектура проекта разделена на следующие модули:

- App-модуль – основной модуль приложения, который связывает в себе все остальные модули и управляет их взаимодействием. Он содержит точку входа в приложение, отвечает за инициализацию зависимостей и обеспечивает связь между Feature-модулями и Core.

- Core-модуль – содержит вспомогательный код, необходимый для работы нескольких Feature-модулей. В Core размещаются общие компоненты, утилиты, обёртки над библиотеками, расширения и вспомогательные функции. Данный модуль не зависит от других модулей, что позволяет использовать его в различных частях приложения без побочных эффектов.

- Feature-модуль – модуль, реализующий конкретную функциональность приложения. Каждый Feature-модуль изолирован от остальных и содержит бизнес-логику, специфичную для своей области. Feature-модуль имеет зависимость только на Core-модуль, что обеспечивает независимость и повторное использование.

На рисунке 1 представлена схема многомодульного подхода, применённого в проекте.

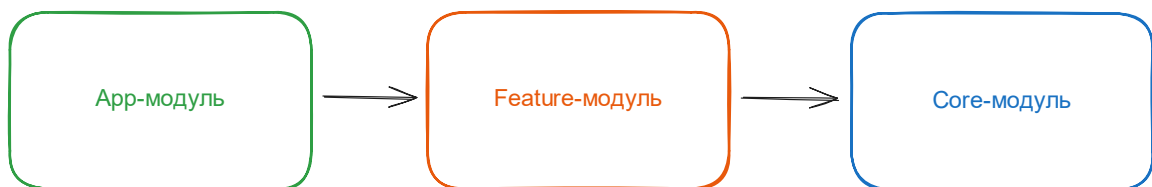


Рисунок 1 – Структура многомодульного приложения

Внутри каждого Feature-модуля реализовано разделение на три слоя в соответствии с принципами Clean Architecture:

- **Presentation** – слой представления, который отвечает за взаимодействие с пользователем и отображение данных. В данном слое реализуются экраны с использованием Jetpack Compose и управляющие ими ViewModel.

- **Domain** – слой бизнес-логики, содержащий интерфейсы репозитория и use-case для обработки данных. Слой не зависит от конкретных технологий хранения и получения данных, что обеспечивает гибкость и тестируемость.

- **Data** – слой доступа к данным, включающий работу с сетью, локальной базой данных и кэшированием. Он реализует интерфейсы, определённые в Domain, обеспечивая прозрачный механизм получения и сохранения данных.

На рисунке 2 представлена схема разделения на слои внутри Feature-модуля.

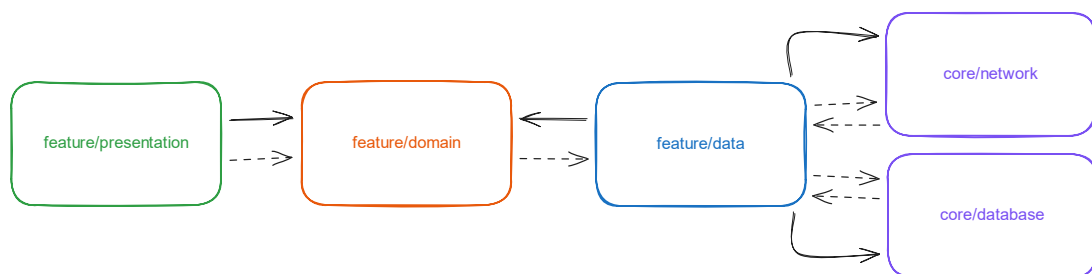


Рисунок 2 – Архитектурное разделение на слои в Feature-модуле  
Для организации взаимодействия между слоями использован паттерн MVVM:

- **Model** – отвечает за работу с данными: хранение, получение и обработку. Model полностью изолирована от View и ViewModel, что упрощает тестирование и повторное использование.

- **ViewModel** – отвечает за подготовку данных для отображения, управляет состоянием пользовательского интерфейса и взаимодействует с

Model. ViewModel выполняет роль посредника между View и Model, предоставляя UI готовые к использованию данные.

– View – реализует пользовательский интерфейс, подписывается на изменения данных из ViewModel и обновляет UI в ответ на эти изменения.

Данный подход минимизирует зависимость между компонентами, упрощает тестирование и сопровождение кода. Комбинация MVVM и Clean Architecture признана оптимальной для разработки сложных Android-приложений, так как позволяет чётко разделить бизнес-логику и представление, делая архитектуру гибкой, масштабируемой и управляемой.

## 2 Разработка навигации

Навигация в приложении реализована с использованием Jetpack Compose Navigation, что обеспечивает маршрутизацию между экранами без применения Fragment и поддерживает Single Activity Architecture. Для повышения типобезопасности используется интерфейс Direction и функция `directionRouteOf`, которая генерирует уникальные строки-маршруты для каждого направления.

Каждый Feature-модуль формирует собственный навигационный граф через DSL `navigationDirection` и `composableDirection`, что изолирует маршруты внутри модуля и облегчает поддержку кода. В качестве примера навигации внутри модуля Profile выделены отдельные направления `ProfileRootDirection` и `ProfileEditDirection`, которые соответствуют экрану профиля и экрану редактирования данных пользователя. Таким образом, при переходе между экранами внутри фичи используется централизованная и типобезопасная маршрутизация (листинг 1).

### Листинг 1 – Пример навигации внутри Feature-модуля Profile

```
fun NavGraphBuilder.navigationProfile() {  
    navigationDirection<ProfileDirection>(start =  
        ProfileRootDirection::class) {  
        composableDirection<ProfileRootDirection> { ProfileScreen() }  
        composableDirection<ProfileEditDirection> { ProfileEditScreen() }  
    }  
}
```

Основной NavHost объединяет все навигационные графы модулей, а доступ к NavController предоставляется через CompositionLocal. Нижняя панель навигации (BottomNavBar) связывает элементы интерфейса с маршрутами, управляет стеком навигации и сохраняет состояние экранов при переключении вкладок. Такой подход минимизирует ошибки при рефакторинге, упрощает добавление новых экранов и обеспечивает согласованную работу навигации как на уровне приложения, так и внутри отдельных фич.

В листинге 2 показано, как объединяются навигационные графы всех Feature-модулей и предоставляется централизованный доступ к контроллеру и фабрике ViewModel через CompositionLocal, что обеспечивает гибкую и масштабируемую архитектуру навигации.

#### Листинг 2 – Основной NavHost приложения

```
@Composable
fun NavHost(
    navController: NavController,
    viewModelFactory: ViewModelProvider.Factory,
    modifier: Modifier = Modifier,
) {
    CompositionLocalProvider(
        LocalViewModelFactory provides viewModelFactory,
        LocalNavController provides navController,
    ) {
        NavHost(
            navController = navController,
            startDestination =
directionRouteOf(AuthorizationDirection::class),
            modifier = modifier,
        ) {
            navigationAuthorization()
            navigationHome()
            navigationSettings()
            navigationProfile()
        }
    }
}
```

### 3 Сетевое взаимодействие и работа с данными

Сетевое взаимодействие в приложении организовано на основе принципов многомодульной архитектуры и разделения ответственности между слоями domain и data. В качестве основного инструмента для работы с HTTP-запросами используется библиотека Retrofit в связке с OkHttp, а для сериализации и десериализации данных Moshi.

В доменном слое модуля core/network определён унифицированный контракт сетевого взаимодействия. Для обработки результатов запросов применяется обобщённая модель ApiResult, представляющая собой sealed-интерфейс с вариантами успешного выполнения запроса и ошибки. Такой подход позволяет централизованно обрабатывать сетевые.

Сетевые источники данных описываются через интерфейсы репозитория, например, UserRemoteRepository и QuotesRemoteRepository, что обеспечивает независимость бизнес-логики от конкретной реализации API. Реализация этих интерфейсов располагается в слое data и использует Retrofit-сервисы для выполнения HTTP-запросов.

Конфигурация сетевого стека выполняется в Dagger-модуле NetworkModule, где создаются и настраиваются OkHttpClient, Retrofit и Moshi.

В листинге 3 показано, как осуществляется обработка успешного и неуспешного сетевого запроса, а также обновление локального кэша.

#### Листинг 3 – Пример обработки сетевого запроса

```
private suspend fun fetchAndCacheQuotes(): List<Quote> {
    return when (val result = remote.getQuotes()) {
        is ApiResult.Success -> {
            local.upsertQuotes(result.data)
            cacheManager.updateCacheTimestamp(CACHE_KEY_QUOTE_LIST)
            result.data
        }
        is ApiResult.Failure -> {
            val cached = local.getQuotes()
            cached.isEmpty { error("Failed to load quotes and no cache
available") }
        }
    }
}
```



## 4 Организация локального хранения данных

Локальное хранение данных в приложении организовано с использованием двух основных механизмов: базы данных Room для структурированных данных и Proto DataStore для хранения пользовательских настроек и простых параметров состояния. Такой подход позволяет эффективно разделить данные по характеру использования и требованиям к их хранению.

Для хранения доменных сущностей, таких как пользователь и список цитат, применяется библиотека Room, обеспечивающая абстракцию над SQLite и поддержку реактивного доступа к данным через Flow. Структура базы данных описана в модуле `core/database/data` и включает таблицы пользователей, цитат и служебную таблицу временных меток кэша. Использование TypeConverter позволяет корректно сохранять сложные типы данных, такие как перечисления и коллекции.

Доступ к локальным данным осуществляется через интерфейсы репозитория доменного слоя (например, `QuotesLocalRepository`), что обеспечивает независимость бизнес-логики от конкретной реализации хранилища. Реализации репозитория инкапсулируют работу с DAO и выполняют преобразование между сущностями базы данных и доменными моделями.

Для управления актуальностью данных реализован компонент `CacheManager`, который хранит временные метки обновления кэша и используется совместно с сетевым слоем. Это позволяет определять, требуется ли повторная загрузка данных из сети, и тем самым снижать количество сетевых запросов и повышать производительность приложения.

Пользовательские настройки и параметры приложения, такие как идентификатор пользователя и выбранная тема оформления, сохраняются с использованием Proto DataStore. В отличие от `SharedPreferences`, данный механизм обеспечивает типобезопасность, атомарность операций и корректную работу с асинхронными потоками данных. Формат данных

описывается в .proto-файле, а доступ к ним осуществляется через репозиторий и use case'ы доменного слоя.

Интеграция локального хранилища с бизнес-логикой наглядно показана в репозитории модуля Home, который объединяет работу с сетевым и локальным источниками данных, а также механизмом кэширования. В листинге 4 показано, как осуществляется выбор между использованием локального хранилища и обновлением данных из сети на основе актуальности кэша. Такой подход обеспечивает устойчивую работу приложения в офлайн-режиме, минимизирует дублирование логики и соответствует принципам Clean Architecture.

Листинг 4 – Пример получения и обновления данных с использованием локального кэша

```
override suspend fun getQuotes(forceUpdate: Boolean): List<Quote> {  
    val isCacheActual = cacheManager.isCacheActual(CACHE_KEY_QUOTE_LIST)  
    return if (!forceUpdate && isCacheActual) {  
        local.getQuotes()  
    } else {  
        fetchAndCacheQuotes()  
    }  
}
```

## 5 Разработка функциональных модулей

В следующих подразделах представлены ключевые модули приложения: авторизация пользователя, профиль пользователя, настройки приложения и лента цитат, с примерами кода из разных слоёв архитектуры.

### 5.1 Авторизация пользователя

Модуль авторизации отвечает за проверку идентичности пользователя и управление его состоянием в приложении. Доменный слой содержит use case LoginUseCase, который инкапсулирует логику входа в систему (Листинг 5).

Листинг 5 – Use Case авторизации пользователя

```
internal class LoginUseCaseImpl @Inject constructor(  
    private val repository: AuthorizationRepository,  
) : LoginUseCase {  
  
    override suspend fun invoke(username: String, password: String) {  
        repository.login(username, password)  
    }  
}
```

Репозиторий `AuthorizationRepositoryImpl` взаимодействует с удалённым сервером и локальной базой данных, обеспечивая синхронизацию пользовательских данных и кэширование (Листинг 6).

### Листинг 6 – Репозиторий авторизации

```
@Singleton
internal class AuthorizationRepositoryImpl @Inject constructor(
    private val local: UserLocalRepository,
    private val remote: AuthRemoteRepository,
    private val cacheManager: CacheManager,
    private val updateUserIdUseCase: UpdateUserIdUseCase,
    private val dispatchers: CoroutineDispatchers,
) : AuthorizationRepository {

    override suspend fun login(username: String, password: String): Unit =
        withContext(dispatchers.io) {
            when (val result = remote.login(username, password)) {
                is ApiResult.Success -> {
                    val remoteUser = result.data
                    val localUser = local.getUser(remoteUser.id)

                    val merged = if (localUser != null &&
                        localUser.isLocallyEdited) {
                        remoteUser.copy(
                            firstName = localUser.firstName,
                            lastName = localUser.lastName,
                            isLocallyEdited = true,
                        )
                    } else {
                        remoteUser.copy(isLocallyEdited = false)
                    }

                    local.upsertUser(merged)
                    cacheManager.updateCacheTimestamp(CACHE_KEY_USER_ME)
                    updateUserIdUseCase(merged.id)
                }

                is ApiResult.Failure -> {
                    error("Failed to load user and no cache available")
                }
            }
        }
}
```

На уровне представления AuthorizationViewModel управляет состоянием экрана, иницирует авторизацию и обрабатывает навигацию (Листинг 7).

### Листинг 7 – ViewModel авторизации

```
internal class AuthorizationViewModel @Inject constructor(  
    private val loginUseCase: LoginUseCase,  
    private val getUserIdUseCase: GetUserIdUseCase,  
    uiStateMapper: AuthorizationUiStateMapper,  
) : BaseViewModel() {  
  
    private val dataState =  
        MutableStateFlow<AuthorizationState>(AuthorizationState.Loading)  
    val uiState: StateFlow<AuthorizationUiState> =  
        dataState.map(uiStateMapper)  
            .stateInViewModel(AuthorizationUiState.Loading)  
    private val _action = Channel<AuthorizationAction>(Channel.BUFFERED)  
    val action: Flow<AuthorizationAction> = _action.receiveAsFlow()  
  
    override fun onFirstInit() {  
        checkAuth()  
    }  
}
```

UI реализован на Compose и адаптируется к состоянию загрузки, успешного входа или ошибке.

## 5.2 Профиль пользователя

Модуль профиля пользователя отвечает за отображение и редактирование персональных данных. GetUserMeUseCase предоставляет данные о текущем пользователе, а ObserveUserMeUseCase позволяет отслеживать изменения через Flow. Репозиторий UserRepositoryImpl обеспечивает синхронизацию данных между локальной базой и сервером, сохраняя изменения пользователя как локально отредактированные (Листинг 8).

### Листинг 8 – Репозиторий пользователя

```
@Singleton  
internal class UserRepositoryImpl @Inject constructor(  
    private val local: UserLocalRepository,  
    private val remote: UserRemoteRepository,  
    private val cacheManager: CacheManager,  
    private val getUserIdUseCase: GetUserIdUseCase,  
    private val dispatchers: CoroutineDispatchers,  
) : UserRepository {  
  
    override suspend fun getUserMe(forceUpdate: Boolean): User =  
        withContext(dispatchers.io) {  
            val userId = getUserIdUseCase() ?: error("User ID not found in  
datastore")  
        }  
    }
```

```

        val isCacheActual = cacheManager.isCacheActual(CACHE_KEY_USER_ME)
        if (!forceUpdate && isCacheActual) {
            local.getUser(userId) ?: fetchAndCacheUser(userId)
        } else {
            fetchAndCacheUser(userId)
        }
    }

    override fun observeUserMeCache(id: Int): Flow<User?> {
        return local.observeUser(id).distinctUntilChanged()
    }

    override suspend fun updateUser(user: User) {
        local.upsertUser(user)
    }

    private suspend fun fetchAndCacheUser(id: Int): User {
        return when (val result = remote.getUserById(id)) {
            is ApiResult.Success -> {
                val remoteUser = result.data
                val localUser = local.getUser(id)

                val merged = if (localUser != null &&
localUser.isLocallyEdited) {
                    remoteUser.copy(
                        firstName = localUser.firstName,
                        lastName = localUser.lastName,
                        isLocallyEdited = true,
                    )
                } else {
                    remoteUser.copy(isLocallyEdited = false)
                }

                local.upsertUser(merged)
                cacheManager.updateCacheTimestamp(CACHE_KEY_USER_ME)
                merged
            }

            is ApiResult.Failure -> {
                local.getUser(id) ?: error("Failed to load user and no cache
available")
            }
        }
    }

    private companion object {

        const val CACHE_KEY_USER_ME = "cache_user_me"
    }

```

ViewModel ProfileViewModel управляет состоянием экрана, подписывается на изменения данных пользователя и инициирует навигацию к редактированию профиля (Листинг 9).

#### Листинг 9 – ViewModel профиля

```
internal class ProfileViewModel @Inject constructor(
    private val getUserMeUseCase: GetUserMeUseCase,
    private val observeUserMeUseCase: ObserveUserMeUseCase,
    uiStateMapper: ProfileUiStateMapper,
) : BaseViewModel() {

    private val dataState =
MutableStateFlow<ProfileState>(ProfileState.Loading)
    val uiState: StateFlow<ProfileUiState> = dataState.map(uiStateMapper)
        .stateInViewModel(ProfileUiState.Loading)
    private val _action = Channel<ProfileAction>(Channel.BUFFERED)
    val action: Flow<ProfileAction> = _action.receiveAsFlow()

    override fun onFirstInit() {
        loadData()
    }

    internal fun onEditButtonClick() {
        viewModelScope.launch {
            _action.send(
                ProfileAction.NavigateTo(ProfileEditDirection::class)
            )
        }
    }
}
```

UI реализован с помощью Jetpack Compose и адаптируется к состоянию загрузки, успешного получения данных или ошибке (Листинг 10).

#### Листинг 10 – UI экрана профиля

```
@Composable
internal fun ProfileScreenContent(
    state: ProfileUiState,
    onEditButtonClick: () -> Unit,
    onRetryClick: () -> Unit,
) {
    when (state) {
        is ProfileUiState.Loading -> {
            LoadingScreen()
        }

        is ProfileUiState.Content -> {
            ProfileContent(state, onEditButtonClick)
        }

        is ProfileUiState.Error -> {
            ErrorScreen(

```

```

        title = state.title,
        description = state.description,
        retryButtonText = state.retryButtonText,
        onRetryClick = onRetryClick,
    )
}
}
}

```

### 5.3 Настройки приложения

Модуль настроек предоставляет пользователю возможность управления параметрами приложения, например, выбор темы оформления. Данные хранятся в `DataStore`, а `ViewModel` подписывается на изменения через `Flow` (Листинг 11).

Листинг 11 – `ViewModel` настроек

```

internal class SettingsViewModel @Inject constructor(
    private val getAppThemeUseCase: GetAppThemeUseCase,
    private val observeAppThemeUseCase: ObserveAppThemeUseCase,
    private val updateAppThemeUseCase: UpdateAppThemeUseCase,
    uiStateMapper: SettingsUiStateMapper,
) : BaseViewModel() {

    private val dataState =
MutableStateFlow<SettingsState>(SettingsState.Loading)
    val uiState: StateFlow<SettingsUiState> = dataState.map(uiStateMapper)
        .stateInViewModel(SettingsUiState.Loading)

    override fun onFirstInit() {
        loadData()
    }

    internal fun onThemeSelect(newTheme: AppTheme) {
        viewModelScope.launch {
            updateAppThemeUseCase(newTheme)
        }
    }
}

```

UI реализован на `Compose` и обновляется автоматически при изменении состояния темы (Листинг 12).

Листинг 12 – UI экрана настроек

```

@Composable
internal fun SettingsScreenContent(
    state: SettingsUiState,
    onThemeSelect: (AppTheme) -> Unit,
    onRetryClick: () -> Unit,
) {

```

```

when (state) {
    is SettingsUiState.Loading -> {
        LoadingScreen()
    }

    is SettingsUiState.Content -> {
        SettingsContent(
            themeOptions = state.themes,
            onThemeSelect = onThemeSelect,
        )
    }

    is SettingsUiState.Error -> {
        ErrorScreen(
            title = state.title,
            description = state.description,
            retryButtonText = state.retryButtonText,
            onRetryClick = onRetryClick,
        )
    }
}
}
}

```

## 5.4 Лента цитат

Модуль ленты цитат отвечает за загрузку, отображение и обновление списка цитат. `GetQuotesUseCase` получает список цитат из репозитория, который объединяет локальный кэш и удалённый источник (Листинг 13).

### Листинг 13 – Репозиторий ленты цитат

```

@Singleton
internal class QuotesRepositoryImpl @Inject constructor(
    private val local: QuotesLocalRepository,
    private val remote: QuotesRemoteRepository,
    private val cacheManager: CacheManager,
    private val dispatchers: CoroutineDispatchers,
) : QuotesRepository {

    override suspend fun getQuotes(forceUpdate: Boolean): List<Quote> =
        withContext(dispatchers.io) {
            val isCacheActual = cacheManager.isCacheActual(CACHE_KEY_QUOTE_LIST)
            if (!forceUpdate && isCacheActual) {
                local.getQuotes()
            } else {
                fetchAndCacheQuotes()
            }
        }

    override suspend fun updateLikeQuote(quoteId: Int, isLiked: Boolean) {
        val quotes = local.getQuotes()
        val updated = quotes.map {
            if (it.id == quoteId) it.copy(isLiked = isLiked) else it
        }
        local.upsertQuotes(updated)
    }
}

```



```

override fun observeQuotesCache(): Flow<List<Quote>> {
    return local.observeQuotes().distinctUntilChanged()
}

private suspend fun fetchAndCacheQuotes(): List<Quote> {
    return when (val result = remote.getQuotes()) {
        is ApiResult.Success -> {
            val remoteQuotes = result.data
            val localQuotes = local.getQuotes().associateBy { it.id }

            val merged = remoteQuotes.map { remote ->
                val local = localQuotes[remote.id]
                if (local != null) {
                    remote.copy(isLiked = local.isLiked)
                } else {
                    remote.copy(isLiked = false)
                }
            }

            local.upsertQuotes(merged)
            cacheManager.updateCacheTimestamp(CACHE_KEY_QUOTE_LIST)
            merged
        }

        is ApiResult.Failure -> {
            val cached = local.getQuotes()
            cached.isEmpty { error("Failed to load quotes and no cache
available") }
        }
    }
}

private companion object {
    const val CACHE_KEY_QUOTE_LIST = "cache_quote_list"
}
}

```

Состояние лайков хранится локально, что позволяет мгновенно обновлять UI без повторных сетевых запросов.

HomeViewModel управляет состоянием экрана, обработкой лайков и обновлением данных (Листинг 14).

#### Листинг 14 – ViewModel ленты цитат

```

internal class HomeViewModel @Inject constructor(
    private val getQuotesUseCase: GetQuotesUseCase,
    private val updateLikeQuoteUseCase: UpdateLikeQuoteUseCase,
    private val refreshQuotesUseCase: RefreshQuotesUseCase,
    private val observeQuotesUseCase: ObserveQuotesUseCase,
    uiStateMapper: HomeUiStateMapper,
) : BaseViewModel() {

    private val dataState = MutableStateFlow<HomeState>(HomeState.Loading)

```

```

val uiState: StateFlow<HomeUiState> = dataState.map(uiStateMapper)
    .stateInViewModel(HomeUiState.Loading)

override fun onFirstInit() {
    loadData()
}

internal fun onLikeClick(quoteId: Int, isLiked: Boolean) {
    viewModelScope.launch {
        runSuspendCatching {
            updateLikeQuoteUseCase(quoteId, isLiked)
        }
        .onFailure {
            Log.e(TAG, "updateLikeQuoteUseCase has failed", it)
        }
    }
}

internal fun onRefresh() {
    viewModelScope.launch {
        updateContentState {
            it.copy(isRefreshing = true)
        }
        runSuspendCatching {
            refreshQuotesUseCase()
        }
        .onFailure {
            Log.e(TAG, "refreshQuotesUseCase has failed", it)
        }
        updateContentState { currentState ->
            currentState.copy(isRefreshing = false)
        }
    }
}
}

```

UI реализован с помощью Compose и поддерживает различные состояния: загрузка, контент или ошибка (Листинг 15).

#### Листинг 15 – UI ленты цитат

```

@Composable
internal fun HomeScreenContent(
    state: HomeUiState,
    onLikeClick: (Int, Boolean) -> Unit,
    onRefresh: () -> Unit,
    onRetryClick: () -> Unit,
) {
    when (state) {
        is HomeUiState.Loading -> {
            LoadingScreen()
        }

        is HomeUiState.Content -> {
            QuoteList(
                state = state,
                onLikeClick = onLikeClick,
            )
        }
    }
}

```

```

        onRefresh = onRefresh,
    )
}

is HomeUiState.Error -> {
    ErrorScreen(
        title = state.title,
        description = state.description,
        retryButtonText = state.retryButtonText,
        onRetryClick = onRetryClick,
    )
}
}
}
}

```

## 6 Реализация фоновых задач и уведомлений

Для обеспечения актуальности контента в приложении была реализована система фоновых задач и уведомлений. Основная цель этого модуля – регулярное обновление списка цитат и информирование пользователя о новых данных. Архитектура решения строится на использовании `WorkManager` для фоновых периодических задач и `NotificationManager` для отображения уведомлений.

Система уведомлений состоит из двух основных компонентов: `QuotesNotificationChannel` для создания канала уведомлений и `QuotesNotificationManager` для генерации и отображения уведомлений (Листинг 16-17). Создание канала уведомлений происходит один раз при запуске приложения, что обеспечивает корректную работу уведомлений на устройствах с Android 8.0 и выше.

### Листинг 16 – Канал уведомлений

```

object QuotesNotificationChannel {

    fun create(context: Context) {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                QuotesNotificationManager.CHANNEL_ID,
                "Updating quotes",
                NotificationManager.IMPORTANCE_DEFAULT,
            ).apply {
                description = "Notifications about quote updates"
            }

            val manager =
                context.getSystemService(Context.NOTIFICATION_SERVICE) as
                NotificationManager

```

```

        manager.createNotificationChannel(channel)
    }
}
}

```

## Листинг 17 – Менеджер уведомлений

```

class QuotesNotificationManager(private val context: Context) {

    fun showQuotesUpdated() {
        val notification = NotificationCompat.Builder(context, CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_launcher_foreground)
            .setContentTitle("Quotes updated")
            .setContentText("We have uploaded fresh quotes!")
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .build()

        val manager =
            context.getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager
        manager.notify(NOTIFICATION_ID, notification)
    }

    companion object {

        const val CHANNEL_ID = "quotes_updates"
        private const val NOTIFICATION_ID = 1001
    }
}

```

Фоновая задача реализована с помощью класса `QuotesRefreshWorker` – специализированного `CoroutineWorker`, который периодически выполняет обновление цитат и инициирует уведомление при успешном завершении работы. `Worker` интегрирован с бизнес-логикой приложения через use case `RefreshQuotesUseCase` и проверку авторизации пользователя через `GetUserIdUseCase`.

Для упрощения создания `Worker`'ов и внедрения зависимостей используется `QuotesWorkerFactory`, а планирование фоновой работы осуществляется через `QuotesWorkerScheduler`. Периодичность обновления установлена в 15 минут с учётом ограничения минимального интервала, поддерживаемого `WorkManager`, и сети (Листинг 18-19).

## Листинг 18 – Фоновый Worker для обновления цитат

```

class QuotesRefreshWorker(
    context: Context,
    params: WorkerParameters,

```

```

private val getUserIdUseCase: GetUserIdUseCase,
private val refreshQuotesUseCase: RefreshQuotesUseCase,
private val notificationManager: QuotesNotificationManager,
) : CoroutineWorker(context, params) {

    override suspend fun doWork(): Result {
        val userId = getUserIdUseCase()
        if (userId == null || userId == 0) {
            return Result.success()
        }

        return try {
            refreshQuotesUseCase()
            notificationManager.showQuotesUpdated()
            Result.success()
        } catch (e: Exception) {
            Result.retry()
        }
    }
}

```

### Листинг 19 – Планирование фонового обновления

```

object QuotesWorkerScheduler {

    private const val WORK_NAME = "quotes_refresh_worker"

    fun schedule(context: Context) {
        val constraints = Constraints.Builder()
            .setRequiredNetworkType(NetworkType.CONNECTED)
            .build()

        val request = PeriodicWorkRequestBuilder<QuotesRefreshWorker>(
            15,
            TimeUnit.MINUTES,
        )
            .setConstraints(constraints)
            .build()

        WorkManager.getInstance(context).enqueueUniquePeriodicWork(
            WORK_NAME,
            ExistingPeriodicWorkPolicy.KEEP,
            request,
        )
    }
}

```

Взаимодействие с MainActivity включает запрос разрешений на отправку уведомлений и инициализацию планировщика фоновых задач. Такой подход обеспечивает автоматическое обновление цитат и уведомление пользователя без вмешательства с его стороны.

## ЗАКЛЮЧЕНИЕ

В работе была достигнута цель – разработка Android-приложения для управления пользовательскими данными и обновления списка цитат с поддержкой офлайн-режима, с соблюдением принципов модульности и разделения ответственности.

В процессе реализации были выполнены ключевые задачи:

- Многомодульная архитектура с разделением на App, Core и Feature-модули и слоями Presentation, Domain и Data обеспечила масштабируемость и изоляцию функционала.
- Навигация на Jetpack Compose с типобезопасными маршрутами и централизованным NavHost упростила переходы между экранами.
- Сетевое взаимодействие и кэширование через Retrofit, ApiResult и локальное хранилище Room обеспечили стабильную работу при нестабильном соединении.
- Локальное хранение и настройки с Room и Proto DataStore позволили поддерживать офлайн-доступ к данным и сохранять пользовательские предпочтения.
- Функциональные модули (авторизация, профиль, настройки, лента цитат) реализованы с ViewModel, репозиториями и Compose UI, обеспечивая реактивное обновление состояния.
- Фоновые задачи и уведомления через WorkManager и NotificationManager обеспечивают периодическое обновление цитат и информирование пользователя.

Все задачи выполнены, и цель работы достигнута: создано модульное, масштабируемое и функционально полноценное приложение с актуальным контентом и удобной архитектурой для дальнейшего расширения.